



13-DARS · 14 TADAN

Yakuniy loyiha: full-stack "Eslatmalar" ilovasi

Barcha bilimlarni birlashtiramiz: PostgreSQL + Express API + JWT autentifikatsiya + React frontend. Loyihalash, backend, frontend va ularni bog'lash bosqichma-bosqich.

Yuqori · ~4–6 soat amaliyot

Endi barcha bo'laklarni bitta haqiqiy ilovaga yig'amiz. **"Eslatmalar" (Notes)** — foydalanuvchilar ro'yxatdan o'tadi, shaxsiy eslatmalar yaratadi, ularni teglar bilan belgilaydi, qidiradi, mahkamlaydi (pin) va arxivlaydi. Loyiha kichik ko'rinadi, lekin unda real ishdagi barcha muammolar bor: sxema dizayni, API, autentifikatsiya, egalik, validatsiya, frontend holati, xatolar, CORS va proksi.



Qanday o'qish kerak

Bu dars — qo'llanma emas, **amaliy ish**. Har bir faylni o'zingiz yarating va har bosqich oxirida ilovani ishga tushirib tekshiring. Tushunmagan joyingiz bo'lsa — tegishli darsga qayting (havolalar matnda).

1-bosqich: Loyihalash

Funksional talablar

- Ro'yxatdan o'tish, kirish, chiqish; sessiya sahifa yangilanganda saqlanadi.
- Eslatma: sarlavha, matn, rang, teglar, "mahkamlangan" va "arxivlangan" belgilari.
- Eslatmalarni yaratish, tahrirlash, o'chirish (CRUD); faqat **o'z** eslatmalarini ko'rish.
- Matn bo'yicha qidiruv, teg bo'yicha filtr, arxiv bo'limi; mahkamlanganlar tepada.

Arxitektura



API shartnomasi (contract)

Frontend va backend o'rtasidagi "kelishuv"ni oldindan yozib qo'yish — ikki tomonni parallel ishlab chiqish imkonini beradi.

Metod va yo'l	Tana / query	Javob
POST /api/auth/register	{ name, email, password }	201 { user } + cookie
POST /api/auth/login	{ email, password }	200 { user } + cookie
POST /api/auth/logout	—	204
GET /api/auth/me	—	200 { user } / 401
GET /api/notes	?q=&tag=&archived=false	200 { data: Note[] }
POST /api/notes	{ title, body, color, tags[] }	201 Note
PATCH /api/notes/:id	istalgan maydonlar	200 Note / 404
DELETE /api/notes/:id	—	204 / 404
GET /api/tags	—	200 [{ name, count }]

Papka tuzilmasi

Matn

```
notes-app/  
├── server/  
│   ├── package.json  
│   ├── .env / .env.example  
│   ├── db/schema.sql  
│   └── src/  
│       ├── server.js  app.js  config.js  db.js  
│       ├── middleware/  auth.js  validate.js  errors.js  
│       └── routes/      auth.routes.js  notes.routes.js  
└── client/  
    ├── package.json  vite.config.js  index.html  
    └── src/  
        ├── main.jsx  App.jsx  api.js  index.css  
        ├── context/AuthContext.jsx  
        ├── pages/    LoginPage.jsx  NotesPage.jsx  
        └── components/  NoteCard.jsx  NoteEditor.jsx  TagFilter.jsx
```

2-bosqich: Ma'lumotlar bazasi

server/db/schema.sql

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- gen_random_uuid() uchun (PG 13+
da o'rnatilgan)

CREATE TABLE IF NOT EXISTS users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL CHECK (length(name) BETWEEN 2 AND 60),
  email       TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX IF NOT EXISTS users_email_uq ON users (lower(email));

CREATE TABLE IF NOT EXISTS notes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL DEFAULT '' CHECK (length(title) ≤ 200),
  body        TEXT NOT NULL DEFAULT '' CHECK (length(body) ≤ 20000),
  color       TEXT NOT NULL DEFAULT 'default',
  tags        TEXT[] NOT NULL DEFAULT '{}',
  pinned      BOOLEAN NOT NULL DEFAULT false,
  archived    BOOLEAN NOT NULL DEFAULT false,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Eng ko'p so'rov: "foydalanuvchining arxivlanmagan eslatmalari,
mahkamlanganlar va yangilari tepada"
CREATE INDEX IF NOT EXISTS notes_user_list_idx ON notes (user_id, archived,
pinned DESC, updated_at DESC);
CREATE INDEX IF NOT EXISTS notes_tags_idx ON notes USING GIN (tags); --
massiv ichida qidiruv uchun
```

Teglar uchun alohida N:M jadval o'rniga PostgreSQL massivi (`TEXT[]`) tanlandi: teglar faqat bitta foydalanuvchiga tegishli, alohida xususiyatlari yo'q, va GIN indeksi `tags @> ARRAY['ish']` so'rovini tezlashtiradi. Bu — loyihaning ehtiyojiga qarab qilingan ongli tanlov (10-dars).

Terminal

```
docker run -d --name notes-db -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=notes
-p 5432:5432 postgres:17
psql postgres://postgres:secret@localhost:5432/notes -f server/db/schema.sql
```

3-bosqich: Backend (Express API)

Terminal

```
mkdir -p notes-app/server && cd notes-app/server
npm init -y && npm pkg set type=module
npm install express pg bcrypt jsonwebtoken cookie-parser zod helmet morgan
npm pkg set scripts.dev="node --watch --env-file=.env src/server.js"
npm pkg set scripts.start="node src/server.js"
```

server/.env.example

```
PORT=3000
DATABASE_URL=postgres://postgres:secret@localhost:5432/notes
JWT_SECRET=bu-yerga-48-baytlik-tasodifiy-kalit
NODE_ENV=development
```

server/src/config.js

```
const required = (name) => {
  if (!process.env[name]) throw new Error(`.env da ${name} yo'q`);
  return process.env[name];
};

export const config = {
  port: Number(process.env.PORT) || 3000,
  databaseUrl: required('DATABASE_URL'),
  jwtSecret: required('JWT_SECRET'),
  isProd: process.env.NODE_ENV === 'production',
};
```

server/src/db.js

```
import pg from 'pg';
import { config } from './config.js';

export const pool = new pg.Pool({ connectionString: config.databaseUrl });
export const query = (text, params) => pool.query(text, params);
```

server/src/middleware/errors.js

```
export class HttpError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export function notFound(req, res, next) {
  next(new HttpError(404, `Topilmadi: ${req.method} ${req.originalUrl}`));
}

export function errorHandler(err, req, res, _next) {
  if (err.type === 'entity.parse.failed') err = new HttpError(400, 'JSON noto\'g\'ri');
  if (err.code === '22P02') err = new HttpError(404, 'Topilmadi'); // noto'g'ri UUID
  const status = err.status ?? 500;
  if (status ≥ 500) console.error(err);
  res.status(status).json({
    error: {
      message: status ≥ 500 && process.env.NODE_ENV === 'production' ? 'Server xatosi' : err.message,
      ...(err.details && { details: err.details }),
    },
  });
}
```

```
import { HttpError } from './errors.js';

export const validate = (schema, source = 'body') => (req, res, next) => {
  const r = schema.safeParse(req[source]);
  if (!r.success) {
    return next(new HttpError(400, 'Validatsiya xatosi',
      r.error.issues.map(i => ({ field: i.path.join('.'), message: i.message
    })))));
  }
  req.valid = r.data;
  next();
};
```

```
import jwt from 'jsonwebtoken';
import { config } from '../config.js';
import { HttpError } from './errors.js';

export const COOKIE = 'notes_token';

export const cookieOptions = {
  httpOnly: true,
  secure: config.isProd,
  sameSite: 'lax',
  maxAge: 7 * 24 * 60 * 60 * 1000, // 7 kun (soddalik uchun refresh
token'siz)
  path: '/',
};

export const signToken = (user) =>
  jwt.sign({ sub: user.id, name: user.name }, config.jwtSecret, { expiresIn:
'7d', algorithm: 'HS256' });

export function requireAuth(req, res, next) {
  const token = req.cookies[COOKIE];
  if (!token) return next(new HttpError(401, 'Tizimga kiring'));
  try {
    const payload = jwt.verify(token, config.jwtSecret, { algorithms: ['HS256']
});
    req.userId = payload.sub;
    next();
  } catch {
    next(new HttpError(401, 'Sessiya muddati tugagan'));
  }
}
```

```

import { Router } from 'express';
import bcrypt from 'bcrypt';
import { z } from 'zod';
import { query } from '../db.js';
import { validate } from '../middleware/validate.js';
import { HttpError } from '../middleware/errors.js';
import { COOKIE, cookieOptions, signToken, requireAuth } from
'../middleware/auth.js';

const router = Router();
const DUMMY = await bcrypt.hash('dummy', 12);

const registerSchema = z.object({
  name: z.string().trim().min(2, 'Ism kamida 2 harf').max(60),
  email: z.string().trim().toLowerCase().email('Email noto\'g\'ri'),
  password: z.string().min(8, 'Parol kamida 8 belgi').max(72),
});
const loginSchema = z.object({
  email: z.string().trim().toLowerCase().email(),
  password: z.string().min(1),
});

const login = (res, user, status = 200) => {
  res.cookie(COOKIE, signToken(user), cookieOptions);
  res.status(status).json({ user: { id: user.id, name: user.name, email:
user.email } });
};

router.post('/register', validate(registerSchema), async (req, res) => {
  const { name, email, password } = req.valid;
  const hash = await bcrypt.hash(password, 12);
  try {
    const { rows: [user] } = await query(
      'INSERT INTO users (name, email, password_hash) VALUES ($1, $2, $3)
RETURNING id, name, email',
      [name, email, hash]
    );
    login(res, user, 201);
  } catch (err) {
    if (err.code === '23505') throw new HttpError(409, 'Bu email band');
    throw err;
  }
}

```

```

});

router.post('/login', validate(loginSchema), async (req, res) => {
  const { email, password } = req.valid;
  const { rows: [user] } = await query('SELECT * FROM users WHERE lower(email)
= $1', [email]);
  const ok = await bcrypt.compare(password, user?.password_hash ?? DUMMY);
  if (!user || !ok) throw new HttpError(401, 'Email yoki parol noto\'g\'ri');
  login(res, user);
});

router.post('/logout', (req, res) => {
  const { maxAge, ...opts } = cookieOptions;
  res.clearCookie(COOKIE, opts).status(204).end();
});

router.get('/me', requireAuth, async (req, res) => {
  const { rows: [user] } = await query('SELECT id, name, email FROM users WHERE
id = $1', [req.userId]);
  if (!user) throw new HttpError(401, 'Foydalanuvchi topilmadi');
  res.json({ user });
});

export default router;

```

```

import { Router } from 'express';
import { z } from 'zod';
import { query } from '../db.js';
import { validate } from '../middleware/validate.js';
import { HttpError } from '../middleware/errors.js';
import { requireAuth } from '../middleware/auth.js';

const router = Router();
router.use(requireAuth); // - bu router'dagi BARCHA
marshrutlar himoyalangan

const COLORS = ['default', 'yellow', 'green', 'blue', 'pink', 'purple'];
const tag =
z.string().trim().toLowerCase().min(1).max(30).regex(/^[a-zA-Z0-9_-]+$/u,
'Teg: harf, raqam, _ yoki -');

const noteSchema = z.object({
  title: z.string().trim().max(200).default(''),
  body: z.string().max(20000).default(''),
  color: z.enum(COLORS).default('default'),
  tags: z.array(tag).max(10).default([]).transform(t => [...new Set(t)]), //
takrorlarni olib tashlash
  pinned: z.boolean().default(false),
  archived: z.boolean().default(false),
}).refine(n => n.title || n.body, { message: 'Sarlavha yoki matn bo\'lishi
kerak' });

const patchSchema = z.object({
  title: z.string().trim().max(200),
  body: z.string().max(20000),
  color: z.enum(COLORS),
  tags: z.array(tag).max(10).transform(t => [...new Set(t)]),
  pinned: z.boolean(),
  archived: z.boolean(),
}).partial().strict(); // strict - noma'lum maydon (masalan user_id)
yuborilsa xato

const listSchema = z.object({
  q: z.string().trim().max(100).optional(),
  tag: tag.optional(),
  archived: z.enum(['true', 'false']).default('false').transform(v => v ===
'true'),

```

```

});

const COLS = `id, title, body, color, tags, pinned, archived,
              created_at AS "createdAt", updated_at AS "updatedAt"`;

// GET /api/notes?q=&tag=&archived=
router.get('/', validate(listSchema, 'query'), async (req, res) => {
  const { q, tag, archived } = req.valid;
  const params = [req.userId, archived];
  let sql = `SELECT ${COLS} FROM notes WHERE user_id = $1 AND archived = $2`;

  if (q) {
    params.push(`%${q}%`);
    sql += ` AND (title ILIKE ${params.length} OR body ILIKE
    ${params.length})`;
  }
  if (tag) {
    params.push([tag]);
    sql += ` AND tags @> ${params.length}`;
  }
  sql += ` ORDER BY pinned DESC, updated_at DESC LIMIT 500`;

  const { rows } = await query(sql, params);
  res.json({ data: rows });
});

router.post('/', validate(noteSchema), async (req, res) => {
  const n = req.valid;
  const { rows: [note] } = await query(
    `INSERT INTO notes (user_id, title, body, color, tags, pinned, archived)
    VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING ${COLS}`,
    [req.userId, n.title, n.body, n.color, n.tags, n.pinned, n.archived]
  );
  res.status(201).json(note);
});

router.patch('/:id', validate(patchSchema), async (req, res) => {
  const entries = Object.entries(req.valid);
  if (!entries.length) throw new HttpError(400, '0\'zgartirish uchun maydon
  yo\'l\'q');

  // Ustun nomlari – faqat sxema ruxsat bergan kalitlar (strict), qiymatlar –
  parametr'lar
  const set = entries.map(([k], i) => `${k} = ${i + 3}`).join(', ');

```

```

const { rows: [note] } = await query(
  `UPDATE notes SET ${set}, updated_at = now()
   WHERE id = $1 AND user_id = $2 -- 🔒 egalik tekshiruvi
  (IDOR'ga qarshi)
   RETURNING ${COLS}` ,
  [req.params.id, req.userId, ...entries.map(([, v]) => v)]
);
if (!note) throw new HttpError(404, 'Eslatma topilmadi');
res.json(note);
});

router.delete('/:id', async (req, res) => {
  const { rowCount } = await query('DELETE FROM notes WHERE id = $1 AND user_id
= $2', [req.params.id, req.userId]);
  if (!rowCount) throw new HttpError(404, 'Eslatma topilmadi');
  res.status(204).end();
});

export default router;

// Teglar statistikasi alohida router'da
export const tagsRouter = Router().get('/', requireAuth, async (req, res) => {
  const { rows } = await query(
    `SELECT t AS name, COUNT(*)::int AS count
     FROM notes, unnest(tags) AS t
     WHERE user_id = $1 AND NOT archived
     GROUP BY t ORDER BY count DESC, t`,
    [req.userId]
  );
  res.json(rows);
});

```

Diqqat: `WHERE id = $1 AND user_id = $2` — boshqa foydalanuvchining eslatmasini tahrirlashga urinish 403 emas, 404 qaytaradi. Bu ataylab: hujumchiga hatto "bunday ID mavjud" degan ma'lumotni ham bermaymiz.

```
import express from 'express';
import helmet from 'helmet';
import morgan from 'morgan';
import cookieParser from 'cookie-parser';
import authRouter from './routes/auth.routes.js';
import notesRouter, { tagsRouter } from './routes/notes.routes.js';
import { notFound, errorHandler } from './middleware/errors.js';

export const app = express();

app.set('trust proxy', 1); // Nginx/hosting proksisi ortida
to'g'ri IP va HTTPS uchun
app.use(helmet());
app.use(morgan('dev'));
app.use(express.json({ limit: '100kb' }));
app.use(cookieParser());

app.get('/api/health', (req, res) => res.json({ ok: true }));
app.use('/api/auth', authRouter);
app.use('/api/notes', notesRouter);
app.use('/api/tags', tagsRouter);

app.use('/api', notFound);
app.use(errorHandler);
```

server/src/server.js

```
import { app } from './app.js';
import { config } from './config.js';
import { pool } from './db.js';

await pool.query('SELECT 1'); // bazaga ulanishni ishga
tushishda tekshiramiz
const server = app.listen(config.port, () => console.log(`🚀 API:
http://localhost:${config.port}`));

// To'g'ri to'xtash (graceful shutdown) – joriy so'rovlar tugashini kutamiz
for (const signal of ['SIGINT', 'SIGTERM']) {
  process.on(signal, () => {
    console.log(`${signal} – to'xtatilmoqda...`);
    server.close(() => pool.end().then(() => process.exit(0)));
  });
}
```

Backend'ni tekshirish

Terminal

```
cp .env.example .env # va JWT_SECRET ni to'ldiring
npm run dev

# -c / -b : cookie'ni faylga saqlash va yuborish (brauzer kabi)
curl -c jar.txt -X POST localhost:3000/api/auth/register -H 'Content-Type:
application/json' \
  -d '{"name":"Sardor","email":"sardor@mail.uz","password":"parol12345"}'

curl -b jar.txt -X POST localhost:3000/api/notes -H 'Content-Type:
application/json' \
  -d '{"title":"Xaridlar","body":"Non, sut, tuxum","tags":
["uy","bozor"],"color":"yellow"}'

curl -b jar.txt "localhost:3000/api/notes?q=sut"
curl -b jar.txt localhost:3000/api/tags
curl -i localhost:3000/api/notes # cookie'siz → 401
```

4-bosqich: Frontend (React)

Terminal

```
cd .. # notes-app/  
npm create vite@latest client -- --template react  
cd client && npm install && npm install react-router
```

client/vite.config.js

```
import { defineConfig } from 'vite';  
import react from '@vitejs/plugin-react';  
  
export default defineConfig({  
  plugins: [react()],  
  server: {  
    // /api bilan boshlanadigan so'rovlar Express'ga uzatiladi:  
    // brauzer uchun hammasi localhost:5173 – CORS ham, cookie muammosi ham  
    yo'q  
    proxy: { '/api': 'http://localhost:3000' },  
  },  
});
```

```

export class ApiError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export async function api(path, { method = 'GET', body, signal } = {}) {
  const res = await fetch('/api' + path, {
    method,
    signal,
    credentials: 'same-origin',
    headers: body ? { 'Content-Type': 'application/json' } : undefined,
    body: body ? JSON.stringify(body) : undefined,
  });
  if (res.status === 204) return null;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new ApiError(res.status, data.error?.message ?? 'Tarmoq xatosi', data.error?.details);
  return data;
}

export const authApi = {
  me: () => api('/auth/me'),
  login: (body) => api('/auth/login', { method: 'POST', body }),
  register: (body) => api('/auth/register', { method: 'POST', body }),
  logout: () => api('/auth/logout', { method: 'POST' }),
};

export const notesApi = {
  list: (params, signal) => api('/notes?' + new URLSearchParams(params), { signal }),
  create: (note) => api('/notes', { method: 'POST', body: note }),
  update: (id, patch) => api(`/notes/${id}`, { method: 'PATCH', body: patch }),
  remove: (id) => api(`/notes/${id}`, { method: 'DELETE' }),
  tags: () => api('/tags'),
};

```

```
import { createContext, useContext, useEffect, useState } from 'react';
import { authApi } from '../api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [checking, setChecking] = useState(true); // sahifa ochilganda sessiyani tekshirish

  useEffect(() => {
    authApi.me()
      .then(d => setUser(d.user))
      .catch(() => setUser(null))
      .finally(() => setChecking(false));
  }, []);

  const value = {
    user,
    checking,
    login: async (creds) => setUser((await authApi.login(creds)).user),
    register: async (data) => setUser((await authApi.register(data)).user),
    logout: async () => { await authApi.logout(); setUser(null); },
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-router';
import { AuthProvider, useAuth } from '../context/AuthContext';
import LoginPage from '../pages/LoginPage';
import NotesPage from '../pages/NotesPage';

function Protected({ children }) {
  const { user, checking } = useAuth();
  if (checking) return <div className="center">⌚</div>;
  return user ? children : <Navigate to="/login" replace />;
}

function GuestOnly({ children }) {
  const { user, checking } = useAuth();
  if (checking) return null;
  return user ? <Navigate to="/" replace /> : children;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<GuestOnly><LoginPage /></GuestOnly>} />
          <Route path="/" element={<Protected><NotesPage /></Protected>} />
          <Route path="/archive" element={<Protected><NotesPage archived /></Protected>} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

```

```

import { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export default function LoginPage() {
  const { login, register } = useAuth();
  const [mode, setMode] = useState('login'); // 'login' | 'register'
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState(null);
  const [busy, setBusy] = useState(false);

  const onChange = (e) => setForm(f => ({ ...f, [e.target.name]: e.target.value }));

  async function onSubmit(e) {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      mode === 'login'
        ? await login({ email: form.email, password: form.password })
        : await register(form);
      // muvaffaqiyatli – GuestOnly avtomatik "/" ga yo'naltiradi
    } catch (err) {
      setError(err.details?.map(d => d.message).join('. ') || err.message);
    } finally {
      setBusy(false);
    }
  }

  return (
    <main className="auth">
      <form onSubmit={onSubmit} className="auth-card">
        <h1>📖 Eslatmalar</h1>
        <div className="tabs">
          <button type="button" className={mode === 'login' ? 'active' : ''}
            onClick={() => setMode('login')}>Kirish</button>
          <button type="button" className={mode === 'register' ? 'active' : ''}
            onClick={() => setMode('register')}>Ro'yxatdan o'tish</button>
        </div>

        {mode === 'register' && (
          <input name="name" placeholder="Ismingiz" value={form.name} onChange=

```

```

{onChange} required autoComplete="name" />
    })
    <input name="email" type="email" placeholder="Email" value={form.email}
onChange={onChange} required autoComplete="email" />
    <input name="password" type="password" placeholder="Parol (kamida 8
belgi)" value={form.password} onChange={onChange}
        required minLength={mode === 'register' ? 8 : undefined}
        autoComplete={mode === 'login' ? 'current-password' : 'new-
password'} />

    {error && <p className="error" role="alert">{error}</p>}
    <button className="primary" disabled={busy}>
        {busy ? '...' : mode === 'login' ? 'Kirish' : 'Hisob yaratish'}
    </button>
</form>
</main>
);
}

```

```

import { useState } from 'react';

export const COLORS = ['default', 'yellow', 'green', 'blue', 'pink', 'purple'];

// Yangi eslatma yaratish va mavjudini tahrirlash uchun bitta komponent
export default function NoteEditor({ initial, onSave, onCancel }) {
  const [title, setTitle] = useState(initial?.title ?? '');
  const [body, setBody] = useState(initial?.body ?? '');
  const [color, setColor] = useState(initial?.color ?? 'default');
  const [tagsText, setTagsText] = useState(initial?.tags?.join(', ') ?? '');
  const [error, setError] = useState(null);
  const [saving, setSaving] = useState(false);

  async function submit(e) {
    e.preventDefault();
    if (!title.trim() && !body.trim()) return setError('Sarlavha yoki matn yozing');
    const tags = tagsText.split(',').map(t => t.trim().toLowerCase()).filter(Boolean);
    setSaving(true);
    try {
      await onSave({ title, body, color, tags });
      if (!initial) { setTitle(''); setBody(''); setTagsText(''); setColor('default'); }
      setError(null);
    } catch (err) {
      setError(err.details?.map(d => `${d.field}: ${d.message}`).join('; ') || err.message);
    } finally {
      setSaving(false);
    }
  }

  return (
    <form className={`editor note-${color}`} onSubmit={submit}>
      <input placeholder="Sarlavha" value={title} onChange={e => setTitle(e.target.value)} maxLength={200} />
      <textarea placeholder="Eslatma matni..." rows={initial ? 8 : 3} value={body} onChange={e => setBody(e.target.value)} />
      <input placeholder="Teglar: ish, g'oyalar" value={tagsText} onChange={e => setTagsText(e.target.value)} />
      <div className="editor-bar">

```

```

    <div className="colors" role="radiogroup" aria-label="Rang">
      {COLORS.map(c => (
        <button key={c} type="button" role="radio" aria-checked={c ===
color} aria-label={c}
          className={`swatch note-${c} ${c === color ? 'selected' :
''}`} onClick={() => setColor(c)} />
      ))}
    </div>
    {onCancel && <button type="button" onClick={onCancel}>Bekor</button>}
    <button className="primary" disabled={saving}>{saving ? 'Saqlanmoqda...'
: 'Saqlash'}</button>
  </div>
  {error && <p className="error">{error}</p>}
</form>
);
}

```

```

export default function NoteCard({ note, onEdit, onUpdate, onDelete, onTagClick
}) {
  const date = new Date(note.updatedAt).toLocaleString('uz-UZ', { dateStyle:
'medium', timeStyle: 'short' });
  return (
    <article className={`note note-${note.color}`}>
      <button className="pin" title={note.pinned ? 'Bo\'shatish' :
'Mahkamlash'} aria-pressed={note.pinned}
        onClick={() => onUpdate(note.id, { pinned: !note.pinned })}>
        {note.pinned ? '📌' : '📌'}
      </button>
      <div className="note-content" onClick={() => onEdit(note)} role="button"
tabIndex={0}
        onKeyDown={e => e.key === 'Enter' && onEdit(note)}>
        {note.title && <h3>{note.title}</h3>}
        <p>{note.body}</p>
      </div>
      {note.tags.length > 0 && (
        <div className="tags">
          {note.tags.map(t => <button key={t} className="tag" onClick={() =>
onTagClick(t)}>#{t}</button>)}
        </div>
      )}
      <footer>
        <small>{date}</small>
        <span>
          <button onClick={() => onUpdate(note.id, { archived: !note.archived
})} title={note.archived ? 'Arxivdan chiqarish' : 'Arxivlash'}>
            {note.archived ? '📁' : '📁'}
          </button>
          <button onClick={() => onDelete(note)} title="O'chirish">🗑️</button>
        </span>
      </footer>
    </article>
  );
}

```

```

import { useCallback, useEffect, useState } from 'react';
import { NavLink } from 'react-router';
import { useAuth } from '../context/AuthContext';
import { notesApi } from '../api';
import NoteCard from '../components/NoteCard';
import NoteEditor from '../components/NoteEditor';

function useDebounce(value, ms = 300) {
  const [v, setV] = useState(value);
  useEffect(() => { const t = setTimeout(() => setV(value), ms); return () =>
clearTimeout(t); }, [value, ms]);
  return v;
}

export default function NotesPage({ archived = false }) {
  const { user, logout } = useAuth();
  const [notes, setNotes] = useState([]);
  const [tags, setTags] = useState([]);
  const [status, setStatus] = useState('loading'); // 'loading' | 'ready'
  | 'error'
  const [q, setQ] = useState('');
  const [tag, setTag] = useState(null);
  const [editing, setEditing] = useState(null);
  const query = useDebounce(q);

  const loadTags = useCallback(() => notesApi.tags().then(setTags).catch(() =>
{}), []);

  // Filtr o'zgarganda ro'yxatni qayta yuklash; eski so'rovni bekor qilish
  useEffect(() => {
    const ctrl = new AbortController();
    setStatus('loading');
    const params = { archived: String(archived) };
    if (query) params.q = query;
    if (tag) params.tag = tag;
    notesApi.list(params, ctrl.signal)
      .then(({ data }) => { setNotes(data); setStatus('ready'); })
      .catch(err => { if (err.name !== 'AbortError') setStatus('error'); });
    return () => ctrl.abort();
  }, [archived, query, tag]);

  useEffect(() => { loadTags(); }, [loadTags]);

```

```

const sortNotes = (list) =>
  [...list].sort((a, b) => (b.pinned - a.pinned) ||
b.updatedAt.localeCompare(a.updatedAt));

async function create(data) {
  const note = await notesApi.create(data);
  setNotes(n => sortNotes([note, ...n]));
  loadTags();
}

// Optimistik yangilash: interfeys darhol o'zgaradi, xato bo'lsa – orqaga
qaytariladi
async function update(id, patch) {
  const prev = notes;
  setNotes(n => n.map(x => (x.id === id ? { ...x, ...patch } : x)));
  try {
    const saved = await notesApi.update(id, patch);
    setNotes(n => {
      const next = n.map(x => (x.id === id ? saved : x));
      // arxiv holati o'zgargan bo'lsa – joriy ro'yxatdan chiqaramiz
      return sortNotes(next.filter(x => x.archived === archived));
    });
    if ('tags' in patch || 'archived' in patch) loadTags();
  } catch (err) {
    setNotes(prev);
    alert(`Saqlanmadi: ${err.message}`);
  }
}

async function remove(note) {
  if (!confirm(`"${note.title || 'Sarlavhasiz'}" eslatmasi o'chirilsinmi?`))
return;
  const prev = notes;
  setNotes(n => n.filter(x => x.id !== note.id));
  try {
    await notesApi.remove(note.id);
    loadTags();
  } catch (err) {
    setNotes(prev);
    alert(err.message);
  }
}

```

```

async function saveEdit(data) {
  await update(editing.id, data);
  setEditing(null);
}

return (
  <div className="layout">
    <header className="topbar">
      <h1>📖 Eslatmalar</h1>
      <input type="search" placeholder="Qidirish..." value={q} onChange={e =>
setQ(e.target.value)} />
      <span className="user">{user.name}</span>
      <button onClick={logout}>Chiqish</button>
    </header>

    <aside className="sidebar">
      <NavLink to="/" end>📖 Eslatmalar</NavLink>
      <NavLink to="/archive">📁 Arxiv</NavLink>
      <h4>Teglar</h4>
      {tags.length === 0 && <small>Teg yo'q</small>}
      {tags.map(t => (
        <button key={t.name} className={t.name === tag ? 'active' : ''}
onClick={() => setTag(t.name === tag ? null : t.name)}>
          #{t.name} <small>{t.count}</small>
        </button>
      ))}
    </aside>

    <main className="main">
      {!archived && <NoteEditor onSave={create} />}

      {tag && <p className="filter-info">Filtr: <b>#{tag}</b> <button
onClick={() => setTag(null)}>x</button></p>}
      {status === 'loading' && notes.length === 0 && <p
className="muted">Yuklanmoqda...</p>}
      {status === 'error' && <p className="error">Eslatmalarni yuklab
bo'lmadi. Sahifani yangilang.</p>}
      {status === 'ready' && notes.length === 0 && (
        <p className="muted">{query || tag ? 'Hech narsa topilmadi.' :
archived ? 'Arxiv bo\'sh.' : 'Birinchi eslatmangizni yozing 🖋️'}</p>
      )}

      <section className="grid">
        {notes.map(note => (

```

```

        <NoteCard key={note.id} note={note} onEdit={setEditing} onUpdate=
{update} onDelete={remove} onTagClick={setTag} />
      )}
    </section>
  </main>

  {editing && (
    <dialog open className="modal" onKeyDown={e => e.key === 'Escape' &&
setEditing(null)}>
      <NoteEditor initial={editing} onSave={saveEdit} onCancel={() =>
setEditing(null)} />
    </dialog>
  )}
</div>
);
}

```

```

:root { --bg: #f8fafc; --card: #fff; --text: #0f172a; --muted: #64748b; --
border: #e2e8f0; --primary: #4f46e5; }
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: var(--bg);
color: var(--text); }
button { font: inherit; cursor: pointer; border: 1px solid var(--border);
background: var(--card); border-radius: 8px; padding: 6px 10px; }
button.primary { background: var(--primary); color: #fff; border-color: var(--
primary); }
input, textarea { font: inherit; width: 100%; padding: 10px; border: 1px solid
var(--border); border-radius: 8px; background: transparent; }
.error { color: #b91c1c; } .muted { color: var(--muted); } .center { display:
grid; place-items: center; height: 100vh; }

/* Auth */
.auth { min-height: 100vh; display: grid; place-items: center; }
.auth-card { width: min(380px, 92vw); display: grid; gap: 12px; background:
var(--card); padding: 28px; border-radius: 16px; box-shadow: 0 10px 40px rgb(0
0 0 / .06); }
.tabs { display: flex; gap: 6px; } .tabs button { flex: 1; } .tabs .active {
border-color: var(--primary); color: var(--primary); }

/* Layout */
.layout { display: grid; grid-template: "top top" auto "side main" 1fr / 220px
1fr; min-height: 100vh; }
.topbar { grid-area: top; display: flex; gap: 12px; align-items: center;
padding: 10px 20px; background: var(--card); border-bottom: 1px solid var(--
border); position: sticky; top: 0; z-index: 5; }
.topbar h1 { font-size: 1.2rem; margin: 0; white-space: nowrap; } .topbar input
{ max-width: 480px; }
.user { margin-left: auto; color: var(--muted); white-space: nowrap; }
.sidebar { grid-area: side; display: flex; flex-direction: column; gap: 4px;
padding: 16px; }
.sidebar a { padding: 8px 10px; border-radius: 8px; color: inherit; text-
decoration: none; }
.sidebar a.active, .sidebar button.active { background: #eef2ff; color: var(--
primary); }
.sidebar button { text-align: left; border: 0; background: none; display: flex;
justify-content: space-between; }
.main { grid-area: main; padding: 20px; }

/* Notes */

```

```

.editor { display: grid; gap: 8px; max-width: 600px; margin: 0 auto 24px;
padding: 14px; border-radius: 14px; border: 1px solid var(--border); box-
shadow: 0 2px 10px rgb(0 0 0 / .04); }
.editor input, .editor textarea { border: 0; } .editor textarea { resize:
vertical; }
.editor-bar { display: flex; gap: 8px; align-items: center; } .colors {
display: flex; gap: 4px; margin-right: auto; }
.swatch { width: 24px; height: 24px; border-radius: 50%; padding: 0; }
.swatch.selected { outline: 2px solid var(--text); }
.grid { columns: 260px; column-gap: 16px; } /* "masonry" uslubidagi
ustunlar */
.note { break-inside: avoid; margin-bottom: 16px; padding: 14px; border-radius:
14px; border: 1px solid var(--border); position: relative; }
.note h3 { margin: 0 0 6px; padding-right: 28px; } .note p { white-space: pre-
wrap; margin: 0; max-height: 240px; overflow: hidden; }
.note-content { cursor: pointer; }
.pin { position: absolute; top: 8px; right: 8px; border: 0; background: none; }
.note footer { display: flex; justify-content: space-between; align-items:
center; margin-top: 10px; color: var(--muted); }
.note footer button { border: 0; background: none; opacity: .6; } .note:hover
footer button { opacity: 1; }
.tags { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 8px; }
.tag { font-size: 12px; padding: 2px 8px; border-radius: 999px; background:
rgb(0 0 0 / .06); border: 0; }
.note-default { background: var(--card); } .note-yellow { background: #fef9c3;
} .note-green { background: #dcfce7; }
.note-blue { background: #dbeafe; } .note-pink { background: #fce7f3; } .note-
purple { background: #ede9fe; }
.modal { position: fixed; inset: 0; margin: auto; border: 0; border-radius:
16px; padding: 0; width: min(640px, 94vw); box-shadow: 0 20px 80px rgb(0 0 0 /
.3); }
.modal .editor { margin: 0; border: 0; }
@media (max-width: 760px) {
  .layout { grid-template: "top" auto "side" auto "main" 1fr / 1fr; }
  .sidebar { flex-direction: row; overflow-x: auto; } .sidebar h4 { display:
none; }
  .topbar { flex-wrap: wrap; } .topbar input { order: 3; max-width: none; }
}

```

5-bosqich: Ishga tushirish va tekshirish

Terminal

```
# 1-terminal — baza (agar ishlayotgan bo'lsa)
docker start notes-db

# 2-terminal — backend
cd notes-app/server && npm run dev

# 3-terminal — frontend
cd notes-app/client && npm run dev
# → http://localhost:5173
```

Qo'lda test ssenariysi

1. Ro'yxatdan o'ting → eslatmalar sahifasiga avtomatik o'tasiz.
2. Teglar bilan 3–4 eslatma yarating; bittasini mahkamlang — u tepaga chiqadi.
3. Qidiruvga yozing — 300ms dan keyin natija filtrlanadi (Network panelida so'rovlar sonini kuzating).
4. Teg ustiga bosing — filtr; arxivlang — "Arxiv" bo'limida paydo bo'ladi.
5. Sahifani yangilang — tizimda qolasiz (httpOnly cookie). DevTools → Application → Cookies: `notes_token` bor, lekin konsolda `document.cookie` uni ko'rsatmaydi.
6. Boshqa brauzerda ikkinchi foydalanuvchi bilan kiring — birinchisining eslatmalari ko'rinmaydi.
7. Backend'ni o'chirib, eslatmani mahkamlashga urining — optimistik o'zgarish orqaga qaytadi va xato chiqadi.

6-bosqich: Production uchun yig'ish

Production'da ikkita alohida server shart emas: Express React'ning build qilingan statik fayllarini o'zi bera oladi.

Terminal

```
cd client && npm run build # → client/dist/ (minifikatsiya qilingan HTML, JS, CSS)
```

server/src/app.js (qo'shimcha, errorHandler'dan oldin)

```
import path from 'node:path';

const dist = path.resolve(import.meta.dirname, '../..../client/dist');
app.use(express.static(dist, { maxAge: '1y', index: false }));
// SPA: /api'dan tashqari barcha GET so'rovlarga index.html (React Router o'zi
// hal qiladi)
app.get(/^(!\s*/api).*/ (req, res) => res.sendFile(path.join(dist,
'index.html')));
```

Endi `NODE_ENV=production npm start` — bitta port, bitta origin, bitta deploy. Keyingi darsda uni internetga chiqaramiz.

Loyihani kengaytirish g'oyalari



Keyingi qadamlar (portfolio uchun)

1. **Markdown:** eslatma matnini `marked` + `DOMPurify` bilan formatlangan ko'rsatish (XSS'dan himoyani unutmang!).
2. **Sahifalash:** 500 ta limit o'rniga kursor asosida (`?before=updatedAt`) cheksiz scroll.
3. **Ulashish:** eslatmani boshqa foydalanuvchi bilan "faqat o'qish" huquqi bilan bo'lishish (`note_shares` jadvali, N:M).
4. **Rasm biriktirish:** multer + S3 (9-dars, 3-misol).
5. **Real vaqt:** bir xil akkaunt ikki qurilmada ochiq bo'lsa, o'zgarish WebSocket (socket.io) orqali darhol ko'rinsin.
6. **Testlar:** API uchun supertest, komponentlar uchun Vitest + Testing Library (14-dars).
7. **TypeScript:** server va klientga turlarni qo'shing — umumiy `Note` turi ikkala tomonda.



Xulosa

- Avval loyihalash: talablar, arxitektura, API shartnomasi, sxema — keyin kod.
- Backend: config → db → middleware (auth, validate, errors) → routes; har bir so'rovda egalik (`user_id = $2`).

- Frontend: api qatlami → AuthContext → himoyalangan marshrutlar → sahifalar → komponentlar.
- Vite proksi ishlab chiqishda, Express static production'da — bitta origin, oddiy cookie autentifikatsiyasi.
- Optimistik yangilash, debounce, AbortController — tez va ishonchli interfeys uchun.

☐ **Darsni o'qib bo'ldim**